

دانشگاه تهران
دانشکده فنی
دانشکده مهندسی برق و کامپیوتر



شبکه‌های عصبی و یادگیری عمیق

تمرین ۱

نام و نام خانوادگی
ابوالفضل قندی

۲۳ فروردین ۱۴۰۵

فهرست مطالب

سوال ۱

- ۱
- بخش اول: سوالات تئوری ۱
- زیر بخش اول: پیاده سازی XOR با MADALINE ۱
- زیر بخش دوم: قانون دلتا ۲
- بخش دوم: سوالات پیاده سازی ۶
- زیر بخش اول: بارگذاری داده ها و تابع پیش پردازش ۶
- زیر بخش دوم: پیاده سازی اتوانکدرها ۶
- زیر بخش سوم: آموزش مدل اتوانکدر ۷
- زیر بخش چهارم: پیاده سازی شبکه پیش خور ۷
- زیر بخش پنجم: ارزیابی دو نوع طبقه بند ۸

سوال ۲

- ۱۰
- بخش اول: سوالات تئوری ۱۰
- زیر بخش اول: MLP ۱۰
- زیر بخش دوم: BACKPROPAGATION ۱۱
- بخش دوم: سوالات پیاده سازی ۱۳
- زیر بخش اول: پیش پردازش ۱۳
- زیر بخش دوم: آموزش شبکه عصبی ساده ۱۳
- زیر بخش سوم: آموزش شبکه عصبی پیشرفته ۱۴
- زیر بخش چهارم: تحلیل نتایج ۱۵

۱	توزیع تعداد نمونه‌ها در دسته‌های مجموعه آموزش و آزمون	۶
۲	نمودار مقایسه‌ای خطای بازسازی اتوانکدرها در طول آموزش	۷
۳	نمودار خطای کلاس‌بندی و دقت مدل‌ها روی داده‌های تست	۸
۴	نمودار تغییرات تابع هزینه (MSE) برای داده‌های آموزش و اعتبارسنجی طی ایپاک‌های آموزش در سه مدل مختلف.	۱۵

بخش اول: سوالات تئوری

زیر بخش اول: پیاده‌سازی XOR با MADALINE

(الف) طراحی شبکه MADALINE: برای پیاده‌سازی تابع XOR بر اساس فرمول داده شده، یعنی $(X_1 \vee X_2) \wedge \neg(X_1 \wedge X_2)$ ، به یک شبکه MADALINE با دو نورون در لایه مخفی (برای محاسبه دو عبارت داخل پرانتز) و یک نورون خروجی (برای محاسبه عمل AND) نیاز داریم. با فرض اینکه ورودی‌ها دوقطبی (Bipolar) و متعلق به مجموعه $\{-1, 1\}$ باشند:

نورون اول (پیاده‌سازی OR): باید زمانی خروجی ۱ بدهد که حداقل یکی از ورودی‌ها ۱ باشد.

$$Z_1 = \text{sgn}(X_1 + X_2 + 1) \implies W_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, b_1 = 1$$

نورون دوم (پیاده‌سازی NAND): باید زمانی خروجی ۱- بدهد که هر دو ورودی ۱ باشند.

$$Z_2 = \text{sgn}(-X_1 - X_2 + 1) \implies W_2 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, b_2 = 1$$

نورون خروجی (پیاده‌سازی AND): این یک گیت ثابت است که خروجی‌های Z_1 و Z_2 را ترکیب می‌کند.

$$Y = \text{sgn}(Z_1 + Z_2 - 1)$$

(ب) مقایسه با شبکه MADALINE در مثال ۲.۲۱: شبکه MADALINE در مثال ۲.۲۱،

یک فضای ویژگی دوبعدی (x_1, x_2) را نشان می‌دهد که توسط دو خط به سه ناحیه تقسیم شده است. این دو خط راست دقیقاً متناظر با مرزهای تصمیم‌گیری ایجاد شده توسط دو نورون لایه مخفی (Adaline) در شبکه ما هستند. در شبکه طراحی شده معادلات خطوط تصمیم (یعنی $X_1 + X_2 + 1 = 0$ و $-X_1 - X_2 + 1 = 0$) دو خط موازی را در فضای ویژگی ایجاد می‌کنند. شبکه MADALINE با ترکیب منطقی این خطوط (ناحیه‌های مثبت و منفی شبکه MADALINE در مثال ۲.۲۱)، مسئله‌های غیرخطی را به نواحی قطعه‌قطعه خطی تجزیه و حل می‌کند.

(پ) علت عدم توانایی پرسپترون تک‌لایه: یک پرسپترون تک‌لایه تنها قادر است کلاس‌هایی را از هم جدا کند که به صورت خطی تفکیک‌پذیر باشند؛ یعنی بتوان با یک ابرصفحه (در فضای دوبعدی، یک خط راست منفرد) داده‌های دو کلاس را کاملاً از هم جدا کرد. همانطور که در مثال ۲.۲۱ مشخص است، مقادیر تابع XOR به گونه‌ای در فضا توزیع شده‌اند که با ترسیم هیچ تک‌خطی نمی‌توان کلاس‌های مثبت و منفی را از یکدیگر متمایز کرد و حتماً به دو خط (یا یک شبکه چندلایه) نیاز است.

زیر بخش دوم: قانون دلتا

با توجه به اینکه خروجی‌ها کلاس‌بندی شده‌اند $(t \in \{1, -1\})$ و نرخ یادگیری $\eta = 0.5$ در نظر گرفته شده است، از قانون یادگیری پرسپترون (نسخه گسسته قانون دلتا) استفاده می‌کنیم که به‌روزرسانی آن به صورت زیر است:

$$W_{new} = W_{old} + \eta(t - y)X \quad , \quad b_{new} = b_{old} + \eta(t - y)$$

تابع فعال‌ساز را به این شکل تعریف می‌کنیم که اگر ورودی خالص (net) بزرگتر یا مساوی صفر بود خروجی ۱ و در غیر این صورت -۱ باشد.

داده‌های آموزش:

$$X_1 = [1, 1, 1, 1]^T, \quad t_1 = 1$$

$$X_2 = [-1, 1, -1, -1]^T, \quad t_2 = 1$$

$$X_3 = [1, 1, 1, -1]^T, \quad t_3 = -1$$

$$X_4 = [1, -1, -1, 1]^T, \quad t_4 = -1$$

مقادیر اولیه: $W = [0, 0, 0, 0]^T$ و $b = 0$.

ایپاک (Epoch) اول:

• ورودی X_1 : $y = 1 \Rightarrow net = W^T X_1 + b = 0 \Rightarrow$ مقدار هدف $t = 1$ است. خطا صفر است و وزن‌ها تغییر نمی‌کنند.

• ورودی X_2 : $y = 1 \Rightarrow net = 0 \Rightarrow$ خطا صفر است و وزن‌ها تغییر نمی‌کنند.

• ورودی X_3 : $y = 1 \Rightarrow net = 0 \Rightarrow$ هدف $t = -1$ است. مقدار خطا $\Delta = -1 - 1 = -2$

$$\Delta W = 0.5 \times (-2) \times X_3 = [-1, -1, -1, 1]^T$$

$$\Delta b = -1 \Rightarrow W = [-1, -1, -1, 1]^T, b = -1$$

• ورودی X_4 :

$$net = (-1)(1) + (-1)(-1) + (-1)(-1) + (1)(1) - 1 = 1 \Rightarrow y = 1$$

هدف $t = -1$. خطا $\Delta = -2$

$$\Delta W = -X_4 = [-1, 1, 1, -1]^T$$

$$\Delta b = -1 \Rightarrow W_{new} = [-2, 0, 0, 0]^T, b = -2$$

ایپاک دوم:

• ورودی X_1 : $y = -1 \Rightarrow net = -2(1) - 2 = -4 \Rightarrow$ هدف $t = 1$. خطا $\Delta = 2$

$$\Delta W = 0.5(2)X_1 = X_1 \Rightarrow W = [-1, 1, 1, 1]^T, b = -1$$

• ورودی X_2 : $net = (-1)(-1) + 1(1) + 1(-1) + 1(-1) - 1 = -1 \Rightarrow y = -1$
 هدف $t = 1$. خطا $\Delta = 2$.

$$\Delta W = X_2 \Rightarrow W = [-2, 2, 0, 0]^T, b = 0$$

• ورودی X_3 : $net = -2(1) + 2(1) + 0 + 0 + 0 = 0 \Rightarrow y = 1$
 هدف $t = -1$. خطا $\Delta = -2$.

$$\Delta W = -X_3 \Rightarrow W = [-3, 1, -1, 1]^T, b = -1$$

• ورودی X_4 : $net = -3(1) + 1(-1) - 1(-1) + 1(1) - 1 = -3 \Rightarrow y = -1$
 هدف $t = -1$. خطا صفر است.

ایپاک سوم:

• ورودی X_1 : $net = -3(1) + 1(1) - 1(1) + 1(1) - 1 = -3 \Rightarrow y = -1$
 هدف $t = 1$. خطا $\Delta = 2$.

$$\Delta W = X_1 \Rightarrow W = [-2, 2, 0, 2]^T, b = 0$$

• ورودی X_2 : $net = -2(-1) + 2(1) + 0 - 2(1) + 0 = 2 \Rightarrow y = 1$
 هدف $t = 1$. خطا صفر است.

• ورودی X_3 : $net = -2(1) + 2(1) + 0 - 2(1) + 0 = -2 \Rightarrow y = -1$
 هدف $t = -1$. خطا صفر است.

• ورودی X_4 : $net = -2(1) - 2(1) + 0 + 2(1) + 0 = -2 \Rightarrow y = -1$
 هدف $t = -1$. خطا صفر است.

ایپاک چهارم (بررسی همگرایی): با اعمال وزن‌های جدید $W = [-2, 2, 0, 2]^T$ و بایاس $b = 0$ ، خروجی شبکه برای تمام داده‌ها را مجدد بررسی می‌کنیم:

$$\bullet \quad X_1 \Rightarrow net = 2 \geq 0 \Rightarrow y = 1 \quad (\text{صحیح})$$

$$\bullet \quad X_2 \Rightarrow net = 2 \geq 0 \Rightarrow y = 1 \quad (\text{صحیح})$$

$$\bullet \quad X_3 \Rightarrow net = -2 < 0 \Rightarrow y = -1 \quad (\text{صحیح})$$

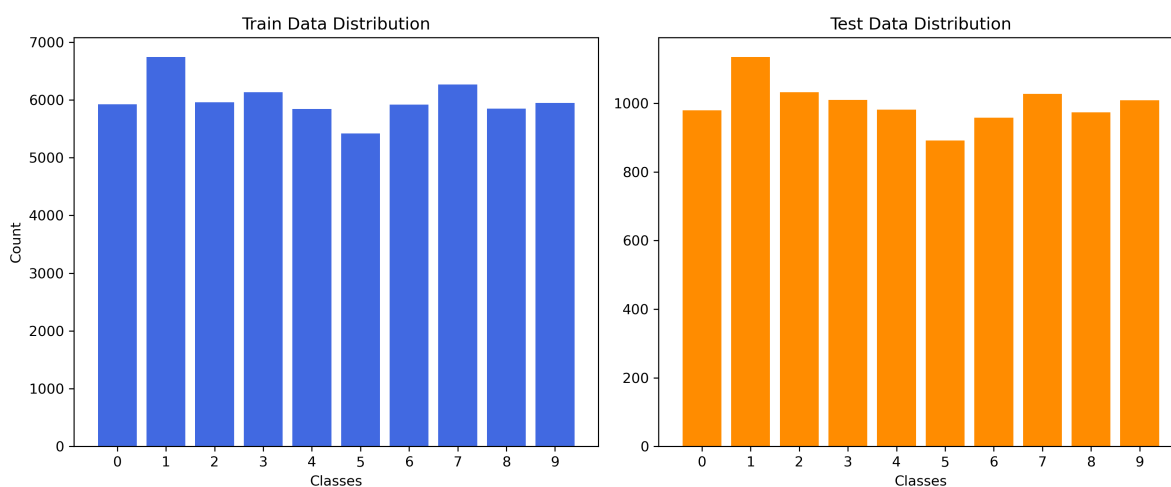
$$\bullet \quad X_4 \Rightarrow net = -2 < 0 \Rightarrow y = -1 \quad (\text{صحیح})$$

از آنجا که خروجی شبکه برای تمام بردارهای آموزشی با مقادیر هدف برابر است، شبکه کاملاً همگرا شده است. بردار وزن نهایی: $W = [-2, 2, 0, 2]^T$ و بایاس نهایی: $b = 0$

بخش دوم: سوالات پیاده‌سازی

زیربخش اول: بارگذاری داده‌ها و تابع پیش‌پردازش

داده‌های آموزشی و آزمون MNIST بارگذاری شدند. بررسی توزیع فراوانی کلاس‌ها (ارقام ۰ تا ۹) در شکل ۱ نشان می‌دهد که مجموعه داده تا حد مطلوبی متوازن است و هیچ کلاسی دچار کمبود شدید نمونه نیست. این توازن از بروز سوگیری (Bias) در فرآیند استخراج ویژگی جلوگیری می‌کند.



شکل ۱: توزیع تعداد نمونه‌ها در دسته‌های مجموعه آموزش و آزمون

در مرحله پیش‌پردازش، تصاویر به نوع float32 تبدیل شده و با تقسیم بر عدد ۲۵۵/۰ به بازه استاندارد [۰, ۱] نرمال‌سازی شدند. سپس، تصاویر ۲۸ در ۲۸ پیکسلی به بردارهای خطی یک‌بعدی با اندازه ۷۸۴ (Flatten) تبدیل گردیدند.

زیربخش دوم: پیاده‌سازی اتوانکدرها

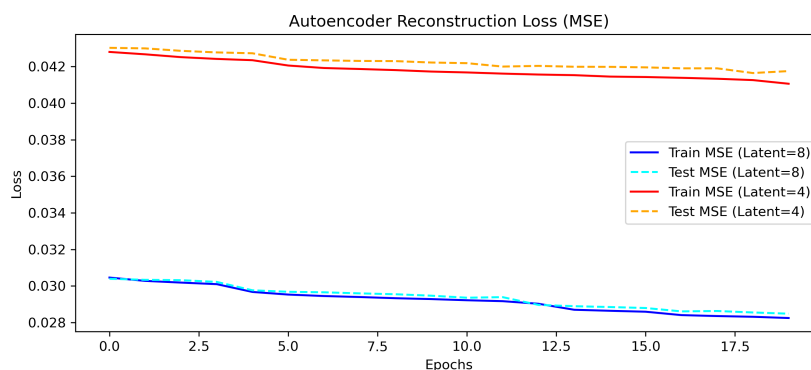
مطابق با خواسته مسئله، دو معماری اتوانکدر با استفاده از کتابخانه PyTorch طراحی شد. تفاوت این دو مدل منحصراً در تعداد نورون‌های لایه فضای نهان (Latent Space) است (یکی با ۸ نورون و دیگری با ۴ نورون).

تحلیل و مقایسه دو شبکه اتوانکدر: بررسی خروجی‌های آموزش نشان می‌دهد که خطای بازسازی (MSE Loss) در ایپاک پایانی (ایپاک ۱۵) برای اتوانکدر با فضای نهان ۸ بُعدی برابر با ۰/۰۳۰۶ و برای اتوانکدر با فضای نهان ۴ بُعدی برابر با ۰/۰۴۲۹ بوده است. دلیل این اختلاف

مشهود، مفهوم گلوگاه اطلاعات (Information Bottleneck) است. کاهش ابعاد از ۷۸۴ به ۴ یک فشرده‌سازی بسیار شدید (نسبت ۱۹۶ : ۱) است که شبکه را مجبور می‌کند بخش زیادی از اطلاعات ساختاری تصویر را دور بریزد. در مقابل، مدل ۸ نورونی ظرفیت بالاتری (Capacity) برای انتقال اطلاعات ورودی به خروجی دارد و به همین دلیل توانسته به خطای بازسازی کمتری دست یابد.

زیربخش سوم: آموزش مدل اتوانکدر

برای ارزیابی دقیق‌تر، اتوانکدرها به مدت ۲۰ اپیک تحت آموزش قرار گرفتند. نمودار روند کاهش خطای میانگین مربعات (MSE) برای هر دو مدل در شکل ۲ ترسیم شده است.



شکل ۲: نمودار مقایسه‌ای خطای بازسازی اتوانکدرها در طول آموزش

همان‌طور که از نمودار پیداست، خطای مدل ۸ نورونی (نمودارهای آبی و فیروزه‌ای) همواره پایین‌تر از مدل ۴ نورونی (نمودارهای قرمز و نارنجی) قرار دارد و سریع‌تر به پایداری می‌رسد. عدم وجود اختلاف فاحش بین خطای آموزش و تست در هر دو مدل، نشان‌دهنده عدم وقوع بیش‌برازش (Overfitting) در فرآیند استخراج ویژگی است.

زیربخش چهارم: پیاده‌سازی شبکه پیش‌خور

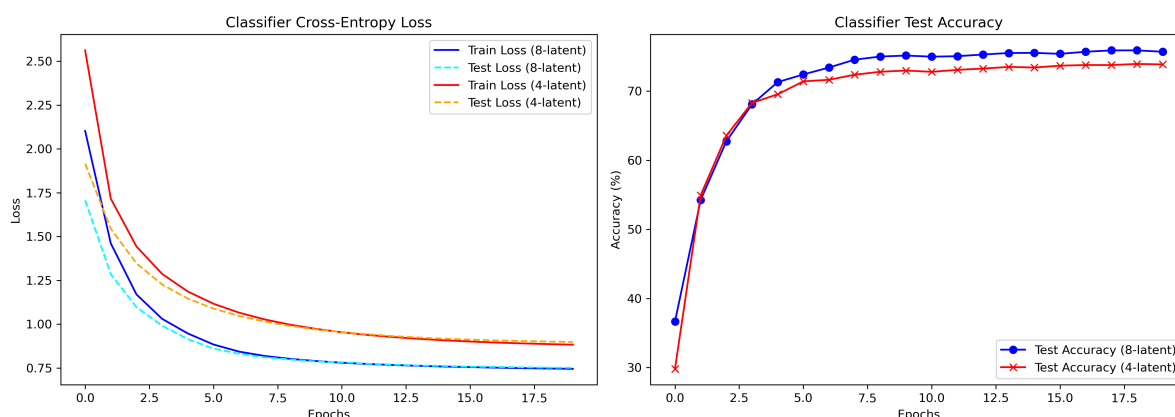
در این مرحله، بخش انکدر از هر دو مدل قبلی جدا شده و به عنوان استخراج‌کننده ویژگی برای شبکه‌های پیش‌خور جدید مورد استفاده قرار گرفت. پارامترهای انکدر فریز شدند (Frozen) تا در حین آموزش طبقه‌بند تغییری نکنند. معماری طبقه‌بند برای انکدر ۸ نورونی شامل لایه‌های ۱۰ → ۴ → ۸ و برای انکدر ۴ نورونی شامل یک لایه ۱۰ → ۴ در نظر گرفته شد.

زیربخش پنجم: ارزیابی دو نوع طبقه‌بند

تعداد پارامترها: با توجه به فریز شدن بخش انکدر، تعداد پارامترهای آموزش‌پذیر (Trainable Parameters) برای هر مدل استخراج شد:

- طبقه‌بند مبتنی بر انکدر ۸ نورونی: ۸۶ پارامتر آموزش‌پذیر.
- طبقه‌بند مبتنی بر انکدر ۴ نورونی: ۵۰ پارامتر آموزش‌پذیر.

نمودار تغییرات تابع خطای Cross-Entropy و دقت روی داده‌های تست در شکل ۳ آورده شده است.



شکل ۳: نمودار خطای کلاس‌بندی و دقت مدل‌ها روی داده‌های تست

تحلیل عملکرد طبقه‌بندها:

- کدام طبقه‌بند بهتر عمل کرد؟ مطابق شکل ۳، طبقه‌بند سوار شده بر روی انکدر ۸ نورونی عملکرد بهتری داشته و به دقت بالاتری (حدود ۷۵٪) نسبت به مدل ۴ نورونی دست یافته است. فضای ۸ بعدی ویژگی‌های تفکیک‌پذیرتری را برای لایه‌های تصمیم‌گیرنده فراهم کرده است.

- تأثیر کاهش ابعاد بر دقت نهایی: بله، کاهش شدید ابعاد به دقت نهایی آسیب جدی وارد کرده است. شبکه‌های پیش‌خور استاندارد روی دیتاست MNIST به راحتی به دقت‌های بالای ۹۵٪ می‌رسند. فشردگی ۷۸۴ پیکسل به تنها ۴ یا ۸ عدد، باعث از دست رفتن بخش عمده‌ای از اطلاعات مکانی تصویر شده که سقف دقت مدل را به شدت محدود کرده است.

● روش‌های پیشنهادی برای بهبود طبقه‌بند:

۱. تنظیم ظریف (Fine-Tuning): پس از آموزش اولیه، وزن‌های انکدر را از حالت فریز خارج کرده و کل شبکه (انکدر + طبقه‌بند) را با نرخ یادگیری کوچک و به صورت یکپارچه (End-to-End) با خطای Cross-Entropy مجدداً آموزش دهیم. این کار فضای نهان را برای کارِ کلاس‌بندی بهینه‌تر می‌کند.
۲. استفاده از معماری کانولوشنی (CNN): جایگزینی لایه‌های تمام‌متصل (Dense) در انکدر با لایه‌های کانولوشنی. شبکه‌های CNN ویژگی‌های محلی (نظیر لبه‌ها) را بدون تخریب ساختار دوبعدی تصویر استخراج می‌کنند که منجر به ایجاد یک فضای نهان بسیار غنی‌تر برای طبقه‌بندی می‌شود.

بخش اول: سوالات تئوری

زیربخش اول: MLP

بررسی تفکیک پذیری خطی: با بررسی مختصات بردارهای ویژگی، می‌توانیم نحوه توزیع آن‌ها را در یک فضای دوبعدی تجسم کنیم. نقاط کلاس اول (w_1) عمدتاً در دو ناحیه متمرکز شده‌اند: یکی در اطراف مبدأ و در سمت چپ-بالا (مانند $[0/2, 0/15]^T$) و دیگری در ربع اول و در سمت راست-بالا (مانند $[1/2, 1/1]^T$). در مقابل، نقاط کلاس دوم (w_2) در نواحی مقابل این دو دسته قرار دارند؛ یعنی یک دسته در سمت راست-پایین (مانند $[1/1, -0/1]^T$) و دسته دیگر در سمت راست-بالا (مانند $[0/1, 1/2]^T$).

این نوع توزیع داده‌ها بسیار شبیه به مسئله معروف XOR است. در این حالت، امکان رسم یک خط مستقیم (یا یک ابرصفحه در ابعاد بالاتر) که بتواند به طور کامل تمام نقاط کلاس w_1 را در یک سمت و تمام نقاط کلاس w_2 را در سمت دیگر قرار دهد وجود ندارد. بنابراین، نتیجه می‌گیریم که این بردارها به صورت خطی تفکیک پذیر نیستند.

طراحی شبکه و توابع فعال‌ساز: از آنجا که داده‌ها خطی تفکیک پذیر نیستند، یک پرسپترون تک‌لایه پاسخگو نخواهد بود و ما به یک پرسپترون چند لایه (MLP) نیاز داریم. برای این مسئله، یک شبکه عصبی با یک لایه پنهان (شامل حداقل ۲ نورون برای ایجاد دو مرز تصمیم‌گیری) و یک لایه خروجی (شامل ۱ نورون) کافی است.

توابع فعال‌ساز مناسب برای این معماری به شرح زیر است:

- لایه پنهان: تابع فعال‌ساز ReLU.

دلیل: این تابع امروزه استانداردترین انتخاب برای لایه‌های پنهان است زیرا محاسبات آن بسیار ساده است، به مدل خاصیت غیرخطی می‌بخشد و مهم‌تر از همه، از مشکل محو شدن گرادیان (Vanishing Gradient) در حین آموزش شبکه جلوگیری می‌کند.

- لایه خروجی: تابع فعال‌ساز سیگموید (Sigmoid).

دلیل: از آنجایی که مسئله ما یک دسته‌بندی دو کلاسه (Binary Classification) است، استفاده از تابع سیگموئید مقدار خروجی نهایی را در بازه (۰, ۱) فشرده می‌کند. این خروجی می‌تواند مستقیماً به عنوان احتمال تعلق یک نمونه به کلاس اول (w_1) تفسیر شود.

زیربخش دوم: BACKPROPAGATION

الف) تعیین ابعاد ماتریس‌ها و بردارها:

با توجه به اینکه m تعداد نمونه‌ها، n_x تعداد ویژگی‌های ورودی و n_h تعداد نورون‌های لایه پنهان است، ابعاد پارامترها به صورت زیر خواهد بود:

- **لایه اول (پنهان):** رابطه لایه اول $Z_1 = W_1 X + b_1$ است. ماتریس ورودی X ابعاد $n_x \times m$ دارد. خروجی این لایه (Z_1) برای n_h نورون و m نمونه باید دارای ابعاد $n_h \times m$ باشد. بنابراین:

— ابعاد ماتریس وزن W_1 : $n_h \times n_x$

— ابعاد بردار بایاس b_1 : $n_h \times 1$ (که در عملیات جمع روی m ستون Broadcast می‌شود).

- **لایه دوم (خروجی):** رابطه لایه دوم $Z_2 = W_2 a_1 + b_2$ است. ورودی این لایه (a_1) دارای ابعاد $n_h \times m$ است. چون خروجی شبکه باینری است (یک نورون خروجی)، Z_2 باید دارای ابعاد $1 \times m$ باشد. بنابراین:

— ابعاد ماتریس وزن W_2 : $1 \times n_h$

— ابعاد بردار بایاس b_2 : 1×1

ب) محاسبه مشتق تابع هزینه نسبت به Z_2 :

تابع هزینه (Cross-Entropy) به صورت زیر تعریف شده است:

$$J = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]$$

برای محاسبه $\frac{\partial J}{\partial Z_{\tau}}$ از قاعده زنجیره‌ای (Chain Rule) استفاده می‌کنیم:

$$\frac{\partial J}{\partial Z_{\tau}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial Z_{\tau}}$$

ابتدا مشتق J نسبت به $\hat{y}$ را محاسبه می‌کنیم (به فرم برداری):

$$\frac{\partial J}{\partial \hat{y}} = -\frac{1}{m} \left(\frac{Y}{\hat{y}} - \frac{1-Y}{1-\hat{y}} \right) = -\frac{1}{m} \left(\frac{Y(1-\hat{y}) - \hat{y}(1-Y)}{\hat{y}(1-\hat{y})} \right) = -\frac{1}{m} \left(\frac{Y - \hat{y}}{\hat{y}(1-\hat{y})} \right)$$

سپس مشتق تابع فعال‌ساز سیگموئید ($\hat{y} = \sigma(Z_{\tau})$) نسبت به Z_{τ} :

$$\frac{\partial \hat{y}}{\partial Z_{\tau}} = \hat{y}(1-\hat{y})$$

حال این دو عبارت را در هم ضرب می‌کنیم:

$$\frac{\partial J}{\partial Z_{\tau}} = \left[-\frac{1}{m} \left(\frac{Y - \hat{y}}{\hat{y}(1-\hat{y})} \right) \right] \cdot [\hat{y}(1-\hat{y})]$$

با ساده‌سازی کسرها، عبارت نهایی به شکل زیر به دست می‌آید:

$$\frac{\partial J}{\partial Z_{\tau}} = -\frac{1}{m}(Y - \hat{y}) = \frac{1}{m}(\hat{y} - Y)$$

بخش دوم: سوالات پیاده‌سازی

زیربخش اول: پیش‌پردازش

در ابتدا مراحل پیش‌پردازش داده‌ها جهت آماده‌سازی برای ورود به شبکه عصبی صورت گرفت:

- **مدیریت مقادیر گمشده و پرت:** ابتدا سطرهایی که متغیر هدف (Life expectancy) در آن‌ها نامشخص بود حذف شدند، زیرا یادگیری نظارت‌شده بدون برچسب هدف امکان‌پذیر نیست. متغیرهای دسته‌ای (مانند کشور و وضعیت توسعه‌یافتگی) با استفاده از روش One-Hot Encoding به مقادیر عددی تبدیل شدند. برای سایر ویژگی‌های عددی که دارای مقادیر گمشده بودند، از روش جایگذاری با میانه (Median) (SimpleImputer) استفاده شد. دلیل استفاده: میانه بر خلاف میانگین، نسبت به داده‌های پرت (Outliers) مقاومت بیشتری دارد و در دیتاست‌هایی مانند شاخص‌های اقتصادی کشورها که معمولاً دارای چولگی و مقادیر اکسترمیم هستند، تقریب بسیار بهتری از مرکزیت داده ارائه می‌دهد.
- **تقسیم داده‌ها:** داده‌ها با نسبت ۷۰ درصد برای آموزش (Train)، ۱۵ درصد برای اعتبارسنجی (Validation) و ۱۵ درصد برای آزمون (Test) تقسیم شدند. تعداد نمونه‌های به‌دست آمده به شرح زیر است: آموزش: ۲۰۴۸ داده، اعتبارسنجی: ۴۴۰ داده، تست: ۴۴۰ داده. همچنین تعداد کل ویژگی‌ها پس از پیش‌پردازش به ۲۰۲ ویژگی (به دلیل One-Hot Encoding) رسید.
- **نرمال‌سازی داده‌ها:** به دلیل تفاوت در مقیاس ویژگی‌ها (مثلاً GDP با مقادیر بسیار بزرگ و BMI با مقادیر کوچک)، از StandardScaler استفاده شد. میانگین و واریانس تنها بر روی داده‌های آموزش محاسبه گردید (استفاده از fit) و سپس همین پارامترها برای مقیاس‌دهی داده‌های اعتبارسنجی و تست استفاده شدند (استفاده از transform) تا از نشت اطلاعات (Data Leakage) جلوگیری شود.

زیربخش دوم: آموزش شبکه عصبی ساده

دو معماری مختلف MLP طراحی و در پایتورچ پیاده‌سازی گردید. برای هر دو مدل از تابع هزینه MSE و بهینه‌ساز Adam با نرخ یادگیری ۰/۰۰۱ استفاده شد.

- **مدل اول (یک لایه پنهان):** لایه ورودی ← لایه پنهان (۶۴ نورون و ReLU) ← لایه خروجی (۱ نورون، خطی).

- مدل دوم (دو لایه پنهان): لایه ورودی ← لایه پنهان اول (۶۴ نورون، ReLU) ← لایه پنهان دوم (۳۲ نورون، ReLU) ← لایه خروجی (۱ نورون، خطی).

تعداد پارامترها: تعداد کل پارامترهای قابل آموزش (شامل وزن‌ها و بایاس‌ها) عبارتند از:

- مدل اول: ۱۳۰۵۷ پارامتر

- مدل دوم: ۱۵۱۰۵ پارامتر

معیارهای ارزیابی روی داده‌های تست:

- مدل اول: $MSE = ۳/۵۵$, $RMSE = ۱/۸۸$, $R^2 = ۰/۹۵۷۰$

- مدل دوم: $MSE = ۴/۰۲$, $RMSE = ۲/۰۱$, $R^2 = ۰/۹۵۱۳$

زیربخش سوم: آموزش شبکه عصبی پیشرفته

برای بررسی امکان بهبود عملکرد، یک مدل پیشرفته با معماری عمیق‌تر و تکنیک‌های منظم‌سازی طراحی شد. این شبکه دارای ۳ لایه پنهان (با ابعاد ۱۲۸، ۶۴، ۳۲ نورون) است.

تغییرات اعمال شده در این مدل:

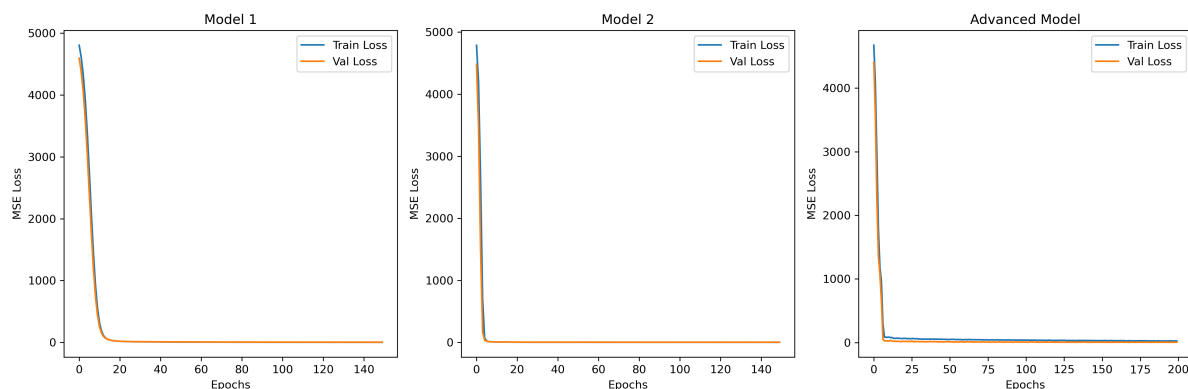
- **Batch Normalization**: پس از لایه‌های خطی برای تسریع همگرایی و پایداری آموزش.

- **Dropout**: با احتمال ۰/۳ و ۰/۲ برای جلوگیری از بیش‌برازش (Overfitting).

- **LeakyReLU**: جایگزین ReLU برای حل مشکل مرگ نورون‌ها.

تعداد پارامترها: ۳۶۷۳۷ پارامتر

نتایج روی داده‌های تست: $MSE = ۳/۵۷$, $RMSE = ۱/۸۹$, $R^2 = ۰/۹۵۶۷$



شکل ۴: نمودار تغییرات تابع هزینه (MSE) برای داده‌های آموزش و اعتبارسنجی طی ایپاک‌های آموزش در سه مدل مختلف.

زیربخش چهارم: تحلیل نتایج

در این بخش به تفسیر معیارهای ارزیابی و مقایسه شبکه‌ها می‌پردازیم:
مفهوم معیارها:

- **MSE (Mean Squared Error):** میانگین مربعات خطای پیش‌بینی را نشان می‌دهد. این معیار خطاهای بزرگ را به شدت جریمه می‌کند.

- **RMSE (Root Mean Squared Error):** جذر MSE است؛ مزیت آن این است که واحد آن دقیقاً مشابه واحد متغیر هدف (در اینجا سالِ امید به زندگی) است. برای مثال، $RMSE \approx 1/88$ در مدل اول یعنی پیش‌بینی‌های مدل ما به طور متوسط حدود ۱/۸ سال با واقعیت اختلاف دارند.

- **R-squared (R^2):** ضریب تعیین است و نشان می‌دهد چه کسری از واریانس متغیر هدف توسط مدل ما توضیح داده شده است. مقدار ۰/۹۵۷۰ به این معناست که بیش از ۹۵ درصد تغییرات امید به زندگی، توسط ویژگی‌های ورودی مدل ما قابل تبیین است که مقدار بسیار مطلوبی است.

مقایسه و نتیجه‌گیری نهایی: با بررسی نتایج، یک مشاهده‌ی بسیار مهم و آموزنده به دست می‌آید: مدل اول (ساده‌ترین مدل با یک لایه پنهان) بهترین عملکرد را ثبت کرده است و با پیچیده‌تر شدن شبکه، دقت مدل روی داده‌های تست کاهش یافته است.

دلیل این امر ماهیت داده‌های مسأله است. داده‌های ما از نوع جدولی (Tabular) هستند و تعداد نمونه‌های آموزشی تنها ۲۰۴۸ مورد است. مدل پیشرفته با داشتن بیش از ۳۶ هزار پارامتر

نسبت به حجم داده‌ها بسیار حجیم است. در شبکه‌های عصبی عمیق روی داده‌های کوچک، مدل به جای یادگیری الگوهای کلی، تمایل به حفظ کردن نویزهای داده‌های آموزش دارد. همچنین، استفاده‌ی همزمان از Dropout با نرخ بالا و Batch Normalization روی داده‌های محدود می‌تواند روند همگرایی شبکه را مختل کند (که نوسانات در نمودار Loss مدل پیشرفته نیز تا حدودی مؤید این موضوع است).

نتیجه می‌گیریم که در مسائل رگرسیون روی شاخص‌های اقتصادی و سلامت، که عموماً فضای ویژگی خطی یا دارای غیرخطی‌های ملایمی هستند، افزایش عمق شبکه نه تنها کمکی نمی‌کند، بلکه باعث افت تعمیم‌پذیری مدل می‌شود. معماری ساده با یک لایه پنهان (مدل اول) تعادل بسیار خوبی (Bias-Variance Tradeoff) برقرار کرده و بالاترین ضریب تعیین (R^2) را به ارمغان آورده است. (۰/۹۵۷۰)